Entregable Lab 03

Ingeniería de Software III - Sistema académico con módulos y Ports

Código en GitHub: https://github.com/keevh/sw3-labs (rama main)

Integrantes: 2191918 Kevin Andres Gallardo Robles
2155160 Cristian Eduardo Johnson Acevedo
2162643 Johan Sebastián Villamizar Ortega

Grupo: G8

Fecha: 28 de octubre de 2026

1. Objetivo y entorno

Seguimos con el proyecto demo-academico del Lab 02. En la parte 1 armamos el flujo Programa, Asignatura, Grupo y Matrícula. En la parte 2 reorganizamos el código por módulos para que Matrícula consulte a Estudiante y a Grupo solo a través de Ports. Los endpoints quedaron iguales después del cambio.

Herramienta	Versión
Java	21
Spring Boot	3.2.4
Base de datos	H2 en memoria, jdbc:h2:mem:demoacademico
Swagger UI (springdoc)	2.6.0
Datos de prueba	JavaFaker 1.0.2

2. Parte 1: flujo funcional

Creamos las entidades Programa, Asignatura, Grupo y Matrícula, cada una con sus DTOs, su repositorio, su service con interfaz, su handler y su controller. Todas las respuestas salen con **ApiResponse**, igual que en el Lab 02.

Recurso	Endpoints
Estudiantes	POST y GET /api/estudiantes, GET /api/estudiantes/pagina, GET /api/estudiantes/buscar?email=, GET, PUT y DELETE /api/estudiantes/{id}
Programas	POST y GET /api/programas, GET, PUT y DELETE /api/programas/{id}
Asignaturas	POST y GET /api/asignaturas, DELETE /api/asignaturas/{id}
Grupos	POST y GET /api/grupos, DELETE /api/grupos/{id}
Matrículas	POST /api/matriculas

La matrícula tiene tres reglas. Si el estudiante o el grupo no existen responde 404. Si el estudiante ya está en ese grupo responde 409. Si el grupo ya llegó a su cupo máximo también responde 409. Además dejamos una restricción única en la base de datos para el par estudiante y grupo.

El seeder con Faker crea 50 estudiantes, 3 programas, 10 asignaturas y 20 grupos al arrancar, y no crea matrículas. Si ya hay datos no los repite.

3. Parte 2: módulos y Ports

Pasamos de carpetas por capa a un paquete por módulo, y lo que usan todos quedó en **common**.

```
co.edu.demoacademico
  common        api, exception, controller (GlobalExceptionHandler)
  config        DataSeeder, DataSeederAcademico
  estudiantes   model, dto, repository, service, handler, controller, exception, port
  programas     model, dto, repository, service, handler, controller
  asignaturas   model, dto, repository, service, handler, controller
  grupos        model, dto, repository, service, handler, controller, port
  matriculas    model, dto, repository, service, handler, controller
```

Port	Paquete	Lo implementa	Método
EstudianteQueryPort	estudiantes.port	EstudianteServiceImpl	Estudiante obtenerPorId(Long id)
GrupoQueryPort	grupos.port	GrupoServiceImpl	Grupo obtenerPorId(Long id)

Con esto **MatriculaServiceImpl** ya no usa EstudianteRepository ni GrupoRepository. Solo recibe los dos Ports y su propio MatriculaRepository. Lo revisé con grep sobre el código del proyecto.

```
$ grep -rn "estudiantes.repository\|grupos.repository"
src/main/java/co/edu/demoacademico/matriculas
(sin resultados)

$ grep -rn "QueryPort" src/main/java/co/edu/demoacademico/matriculas
.../matriculas/service/MatriculaServiceImpl.java:5:import
...estudiantes.port.EstudianteQueryPort;
.../matriculas/service/MatriculaServiceImpl.java:7:import ...grupos.port.GrupoQueryPort;
.../matriculas/service/MatriculaServiceImpl.java:22:    private final EstudianteQueryPort
estudiantePort;
.../matriculas/service/MatriculaServiceImpl.java:23:    private final GrupoQueryPort grupoPort;
```

4. Guion de pruebas en Swagger

Corrimos el guion de la guía en Swagger UI, con la base recién creada. Creamos un programa, una asignatura, un grupo con cupo máximo 2 y un estudiante, y después hicimos las matrículas. Las capturas de abajo son de la versión final por módulos.

Paso	Esperado	Resultado
1 POST programa	201	201
2 POST asignatura con programaId	201	201
3 POST grupo con cupoMax 2	201	201
4 POST estudiante	201	201
5 POST matrícula 1	201	201
6 POST matrícula 2	201	201
7 POST matrícula 3 al mismo grupo	409 sin cupo	409 El grupo no tiene cupo disponible
8 POST matrícula repetida	409 duplicada	409 El estudiante ya está matriculado en ese grupo
Extra, estudiante 99999	404	404 Estudiante no encontrado: 99999

Dos matrículas exitosas

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/matriculas' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "estudianteId": 1,
  "grupoId": 21
}'
```

Request URL

```
http://localhost:8080/api/matriculas
```

Server response

Code	Details
201 *Undocumented*	Response body

```
{
  "success": true,
  "message": "Matrícula registrada",
  "data": {
    "id": 1,
    "estudianteId": 1,
    "estudianteNombre": "Víctor Padrón Sauceda",
    "grupoId": 21,
    "codigoGrupo": "G-IS3-01",
    "fechaRegistro": "2026-10-26T04:00:00.766720523Z"
  },
  "timestamp": "2026-10-26T04:00:00.791088571Z"
}
```

Download



Figura 1. Matrícula 1, estudiante 1 en el grupo 21, responde 201.

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/matriculas' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "estudianteId": 2,
  "grupoId": 21
}'
```

Request URL

```
http://localhost:8080/api/matriculas
```

Server response

Code	Details
201 *Undocumented*	Response body

```
{
  "success": true,
  "message": "Matrícula registrada",
  "data": {
    "id": 2,
    "estudianteId": 2,
    "estudianteNombre": "María Teresa Piña Molina",
    "grupoId": 21,
    "codigoGrupo": "G-IS3-01",
    "fechaRegistro": "2026-10-26T04:00:02.191617690Z"
  },
  "timestamp": "2026-10-26T04:00:02.195669732Z"
}
```

Download



Figura 2. Matrícula 2, estudiante 2 en el mismo grupo, responde 201.

Error por cupo y error por duplicado

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/matriculas' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "estudianteId": 3,
  "grupoId": 21
}'
```

Request URL

```
http://localhost:8080/api/matriculas
```

Server response

Code	Details
409 *Undocumented*	Error: response status is 409 Response body

```
{
  "success": false,
  "message": "El grupo no tiene cupo disponible",
  "data": null,
  "timestamp": "2026-10-26T04:00:03.532973700Z"
}
```

Download



Figura 3. La tercera matrícula en el grupo lleno responde 409.



Figura 4. Repetir la matrícula del estudiante 1 responde 409.

5. Que no haya regresión

Hicimos el mismo guion en las dos versiones del proyecto, la de la parte 1 por capas y la de la parte 2 por módulos, cada una con la base recién creada. Salieron los mismos códigos y los mismos mensajes. Abajo van los dos errores de la parte 1 para compararlos con las figuras 3 y 4.

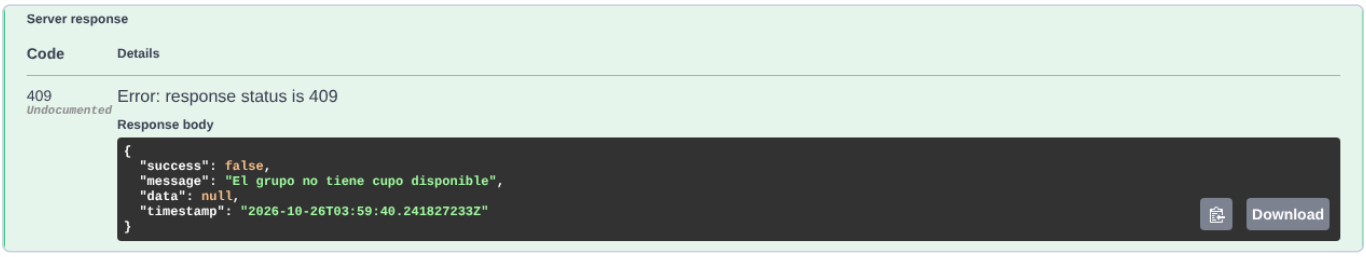


Figura 5. Sin cupo, antes del refactor.



Figura 6. Duplicada, antes del refactor.

También volví a correr el guion con curl sobre el proyecto final, con un build limpio. Dio 201 en los pasos 1 a 6, 409 en cupo y duplicada y 404 con el estudiante que no existe. El seeder dejó 50 estudiantes, 3 programas y 20 grupos.

6. Conteos en H2

Con la consola de H2 contamos las filas de las cinco tablas después del guion. Dan un registro más que el seeder porque el guion creó un programa, una asignatura, un grupo y un estudiante, y quedan las 2 matrículas.

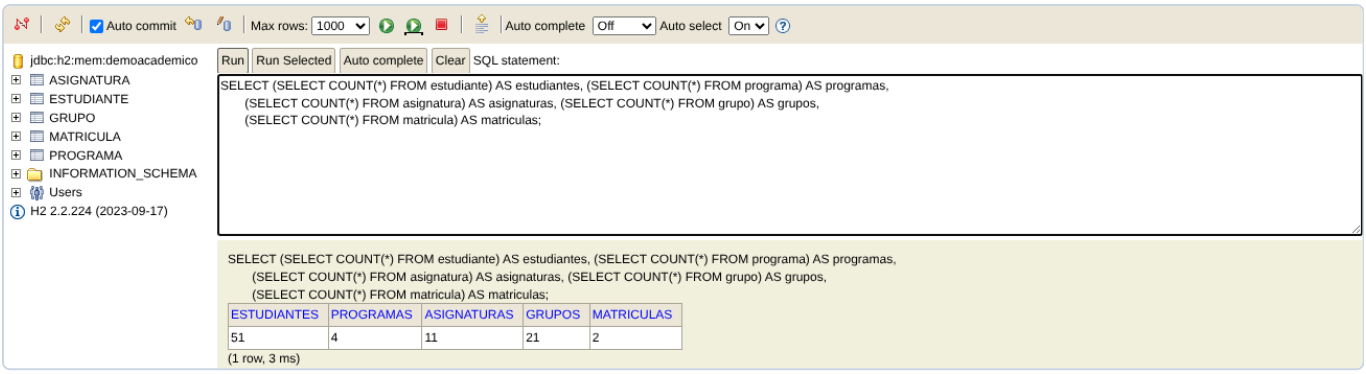


Figura 7. Estudiantes 51, programas 4, asignaturas 11, grupos 21 y matrículas 2.

Anexos

Código: https://github.com/keevh/sw3-labs, rama main. El README del proyecto explica cómo correrlo, las URLs de Swagger y de H2 y los Ports.

Swagger: http://localhost:8080/swagger-ui.html
H2: http://localhost:8080/h2-console